

Hochschule Esslingen

Fakultät für Informatik

Masterarbeit

Prognose individuellen Mobilitätsverhaltens auf Basis eines mit Bewegungsdaten trainierten Machine-Learning-Modells

Achim Baumgärtner

Matrikelnummer: 766010

Erstgutachter: Prof. Dr. Sonntag

Zweitgutachter: Prof. Dr. Schober

Esslingen am Neckar, 15. Juni 2026

Kurzfassung

TODO: Kurzfassung sollte das Thema, die Methodik, die wichtigsten Ergebnisse und die Schlussfolgerungen in ca. 150–300 Wörtern zusammenfassen.

- ziel
- datenbasis
- methode
- ergebnisse
- beitrag

Die Auswertung fahrzeugspezifischer Daten ermöglicht es, bei kontinuierlichen Nutzungsmustern das Bewegungsverhalten vorherzusagen. Dadurch kann bidirektionales Laden effizienter eingesetzt werden, was sowohl die Fahrzeughalter als auch das Stromnetz entlastet. Ziel ist es, durch Machine Learning ein Modell zu entwerfen, das möglichst genau individuelles Verkehrsverhalten prognostizieren kann. Zu bewerkstelligen ist dieses Vorhaben durch eine Ablationsstudie; Es werden auf Basis einer Datengrundlage Modelle erstellt und untereinander verglichen, wodurch die Annahme entsteht, dass ein Modell besser abschneiden wird, als die anderen. Das stärkste Modell wird dann weiter entwickelt um die Ergebnisse zu optimieren und somit die Zuverlässigkeit und den Beitrag zum Stromnetz von elektrischen Fahrzeugen zu erhöhen.

Schlüsselwörter: Schlüsselwort 1, Schlüsselwort 2, Schlüsselwort 3

Abstract

TODO: Insert the English abstract here. The abstract should summarize the topic, methodology, key results, and conclusions in approx. 150–300 words.

Keywords: Keyword 1, Keyword 2, Keyword 3

Inhaltsverzeichnis

Kurzfassung	i
Abstract	ii
Abbildungsverzeichnis	vi
Tabellenverzeichnis	vii
0.1. TODO:	1
0.1.1. Lag	1
0.1.2. Rolling	1
0.1.3. Feature Sicherung	1
0.1.4. Robustheit	1
1. Einleitung	5
1.1. Motivation	5
1.2. Problemstellung	6
1.3. Zielsetzung	7
1.4. Forschungsfragen	7
1.5. Aufbau der Arbeit	7
2. Theoretische Grundlagen	9
2.1. Mobilitätsdaten	9
2.2. Machine Learning	10
2.2.1. Überwachtes Lernen	11
2.2.2. Merkmale und Feature Engineering	11
2.2.3. Training	12
2.2.4. Generalisierung	13
2.2.5. Validierung	14
2.2.6. Bewertungsmetriken	14
2.2.7. Entscheidungsbäume	14
2.2.8. Ensemble-Methoden: Random Forest und Gradient Boosting	15
2.2.9. Unsicherheit und Quantilsregression	15
2.3. Verarbeitung von Mobilitätsdaten	15

2.4. Stand der Forschung	17
2.4.1. Verwandte Arbeiten	18
2.4.2. Mobilitäts- und Nutzungsforecast	19
2.5. Zusammenfassung	19
3. Methodik	20
3.1. Forschungsdesign	20
3.2. Vorgehen	20
3.3. Datenerhebung	20
3.4. Datenanalyse	21
3.5. Zusammenfassung	21
4. Implementierung	22
4.1. Systemarchitektur	22
4.2. Technologieauswahl	22
4.3. Umsetzung	22
4.4. Herausforderungen	22
4.5. Zusammenfassung	22
5. Evaluation	23
5.1. Evaluationsdesign	23
5.2. Testumgebung	23
5.3. Ergebnisse	23
5.4. Zusammenfassung	23
6. Diskussion	24
6.1. Interpretation der Ergebnisse	24
6.2. Vergleich mit verwandten Arbeiten	24
6.3. Limitationen	24
6.4. Implikationen	24
7. Fazit und Ausblick	25
7.1. Zusammenfassung	25
7.2. Beantwortung der Forschungsfragen	25
7.3. Ausblick	25
Literaturverzeichnis	26
A. Anhang	27
A.1. Zusätzliche Abbildungen	27
A.2. Zusätzliche Tabellen	27

A.3. Quellcode	27
Eidesstattliche Erklärung	28

Abbildungsverzeichnis

2.1. Beispielhafte Auszüge zweier Mobilitätsdatensätze.	9
2.2. Teilbereiche des maschinellen Lernens mit beispielhaften Anwendungen (Wuttke 2024).	10
2.3. Simpler binärer Entscheidungsbaum (Schilli 2017)	15
2.4. Entscheidungsbaum mit Aufteilungskriterien (Gini-Index, Stichpro- bengröße und Klassenverteilung pro Knoten) (Schilli 2017)	16
2.5. Generalisierte Pipeline zur Verarbeitung von Mobilitätsdaten.	16

Tabellenverzeichnis

2.1. Charakterisierung von Anpassungen anhand von Fehlerwerten Geron 2022.	14
2.2. Phasen der Mobilitätsdaten-Pipeline	17
2.3. Zentrale Forschungssektionen im Umfeld der individuellen Fahrzeug- nutzungsprognose.	18
2.4. Fünf Fahrertypen des Individualverkehrs nach Ji u. a. (2023).	19

Logs

0.1. TODO:

0.1.1. Lag

0.1.2. Rolling

0.1.3. Feature Sicherung

0.1.4. Robustheit

- Robustheit gegenüber unvollständigen Daten
- Robustheit gegenüber fehlerhaften Daten
- Robustheit gegenüber unregelmäßigen Daten
- Drop von Datenpunkten bei fehlerhaften werten

2026-05-26 — Multi-Trip-Auflösung pro Tag Die bisherige Tageskarte fasste einen Tag auf eine einzige Abfahrt/Rueckkehr-Spanne zusammen (**depEst** = erste aktive Stunde, **retEst** = letzte). Damit gingen die realistischen Tagesmuster eines Nutzers verloren: morgens zur Arbeit, mittags Lunch, abends Freizeit — drei diskrete Trips, im Heatmap-Grid sichtbar, aber im Day-Card unsichtbar.

Eingefuehrt: Backend-Helper `_extract_trips` extrahiert zusammenhaengende Aktiv-Bloেকে aus dem stueundlichen Forecast-Grid (Threshold = adaptive Schwelle aus Fix 2). Jede Day-Card traegt jetzt `trips[]` mit `startH`, `endH`, `durationH`, `peakP`, `meanP` und P10/P90 pro Block. Das Frontend rendert die Trips als horizontale Kartenleiste pro Tag.

Limitierung: die Auflöesung bleibt bei 1 Stunde, weil `data_adapters.py` die Zeitstempel beim Laden auf `.dt.floor("h")` diskretisiert. Sub-Stunden-Präezision waere nur ueber Aenderung der Datenpipeline (raw timestamps fuer routine und VED erhalten) oder durch Interpolation moeglich; letzteres haetten wir abgelehnt, weil es kuenstliche Genauigkeit suggerieren wuerde. UI macht das jetzt transparent (“Auflöesung: 1 Stunde”) statt 5.0 als Zeit-Wert zu zeigen.

2026-05-26 — Drei Fixes gegen Forecast-Kollaps bei niedriger Basisrate Diagnose nach den ersten Hourly-Forecast-Laeufen: nur `routine` liefert sinnvolle Aktiv-Fenster, `emobpy`, `realworlddev` und `ved` produzieren `pUsed=0.0` durchgaengig. Ursache war eine Kombination aus Klassen-Imbalance (Aktiv-Rate 3-10%) und selbst-verstaerkendem autoregressiven Kollaps im Monte-Carlo-Rollout: fruehe Bernoulli-Nullen erzeugen Lag-Werte gleich Null, was die Wahrscheinlichkeit weiter senkt. Drei Aenderungen:

- **Klassengewichtung:** `class_weight="balanced"` im Hourly-Classifer beider Forecaster (RF und LGBM). Hebt die Minderheits-Klassen-Wahrscheinlichkeiten in den kalibrierten Bereich.
- **Adaptive Aggregations-Schwelle:** statt fixem `p_mean ≥ 0.5` jetzt `p_mean ≥ clamp(1.5 · base_rate, 0.30, 0.60)`. Niedrigaktive Fahrzeuge erhalten damit Sinn-Aussagen, hochaktive (Pendler) werden nicht uebersaettigt.
- **Soft-Mode:** optionaler deterministischer Rollout-Pfad. Statt Bernoulli-Sampling wird die kontinuierliche Wahrscheinlichkeit selbst als Lag weitergereicht. Verhindert den Sample-Kollaps, kostet aber die Quantil-Streuung (`P10 = P50 = P90 = mean`). Frontend-Toggle in der Sidebar; Default bleibt Monte-Carlo wegen statistischer Korrektheit.

Methodisch erlaubt der Toggle eine saubere Ablation: *MC* ist die robuste statistische Variante, *Soft* zeigt das Verhalten ohne Sampling-Rauschen. Bei niedrigen Basisraten konvergieren beide auf aehnliche Mittelwerte; bei hohen Basisraten unterscheiden sich nur die Quantile.

2026-05-26 — Auto-regressives Hourly-Forecasting (Hybrid-Pipeline) Der bisherige Day-Forecast lieferte nur ein aggregiertes `pUsed` pro Tag und sagte damit *nicht*, wann ein Fahrzeug genutzt wird — nur *ob* der Tag im Schnitt aktiv ist. Eingefuehrt: `predict_hourly_mc` in `RandomForestForecaster` und `LGBMForecaster`. Beide trainieren ein stuendliches Schwesternmodell auf den gleichen Daten (Features aus `base.py`: `hour`, `weekday`, `is_weekend`, Lags 1/2/24/168, Rolling-Mittel 24 h und 168 h) und persistieren die letzten 168 h des Default-Fahrzeugs als Startpunkt.

Vorhersage erfolgt *auto-regressiv* und *Monte-Carlo*: 100 unabhaengige Sample-Pfade rollen Stunde fuer Stunde vorwaerts, jeweils `Bernoulli(p)` aus der Modell-Wahrscheinlichkeit ziehen und den gezogenen Wert als `lag_1` fuer die naechste Stunde weiterreichen. Pro Forecast-Stunde liefert das einen Mittelwert `p_mean` sowie die Quantile `p10`, `p50`, `p90`. Damit ist die Unsicherheit nicht mehr eine separate Annahme, sondern faellt aus der Sample-Streuung heraus.

Backend (`_to_result`) ist hybrid: Stunden-Grid ist die Quelle der Wahrheit, Tageskarten werden daraus aggregiert (`pUsed` = Anteil Stunden mit `p_mean` $\geq$ 0.5; `depEst` / `retEst` = erste / letzte solche Stunde; P10/P90-Bands aus den entsprechenden Quantil-Schwellen). Legacy-`.joblib` ohne `hourly_clf` fallen auf den alten Day-Pfad zurueck. Frontend zeigt im Stunden-Forecast-Panel eine Heatmap (Forecast-Tag $\times$ Stunde) mit Farbe = `p_mean` und Saettigung = Konfidenz (eng = sicher, blass = breite Sample-Streuung).

Methodisch hat dieser Schritt zwei Konsequenzen: (1) der Vergleich zwischen Datenquellen wird vergleichbar, weil alle nun dieselbe Output-Form haben (P-Vektor mit 168 Eintraegen + Quantile); (2) das “ich weiss nicht” der Modelle wird explizit sichtbar — breite P10/P90 zeigen, wann der Forecast *nicht* vertrauenswuerdig ist, was eine saubere Diskussionsgrundlage liefert.

2026-05-26 — Training-Reiter im Frontend Neuer Tab `/training` neben Forecast und Einstellungen. Listet pro Datenquelle (`emobpy`, `realworlddev`, `ved`, `routine`) und Algorithmus (`rf`, `lgbm`) eine deskriptive Auswahl, plus Eingaben fuer `history_days` und (wo sinnvoll) `limit_vehicles`. Triggert POST `/api/train`, das `train_<source>_forecaster.py` als Subprocess startet; Log-Output wird via Polling auf GET `/api/train/{job_id}` in Echtzeit angezeigt. Damit kann die Ablationsstudie auch ohne Terminal-Zugriff ausgefuehrt werden, was den Dokumentationsaufwand fuer die Methodik-Sektion reduziert (jeder Lauf hat Job-ID + erhaltener Log).

2026-05-26 — Modul-Reorganisation in `forecast/` und `classifier/` Die beiden Modellfamilien (binärer Driving-Classifer vs. 1-7-Tage-Forecaster) wurden in eigene Unterordner unter `code/model_scripts/` verschoben. Geteilte Helpers (`base.py`, `data_adapters.py`) bleiben oben. Begründung: klare Trennung der Forschungsschienen — die Cross-Validation- Methodik (Datenqualität, binär) hängt nicht mit der Anwendungs-Schiene (Tages-Forecast, multi-vehicle, Quantile) zusammen.

2026-05-26 — Ablations-Registry und `LGBMForecaster` Eingeführt: `forecaster_registry.py` mit Lazy-Import-Lookup `name` $\rightarrow$ Klasse. Erste Schwesterimplementierung `LGBMForecaster` mit nativer Quantil-Regression (drei separate `LGBMRegressor`en mit `objective="quantile"`, `alpha=0.1/0.5/0.9`) statt RF-Tree-Variance. Damit ist die Ablationsstudie (LogReg $\rightarrow$ RF $\rightarrow$ LightGBM $\rightarrow$ LSTM $\rightarrow$ TFT) erweiterbar, ohne Backend, Trainings-skripte oder Frontend zu ändern. Backend-Schnittstelle auf `predict_proba_day` + `predict_quantiles` abstrahiert — keine Direkt-Inspektion von `reg.estimators_` mehr.

2026-05-26 — Multi-Vehicle-Forecaster `RandomForestForecaster.fit` akzeptiert jetzt einen `vehicle_id`-mehrwertigen Eingabe-Frame. Trainingszeilen kommen aus allen Fahrzeugen (Lag-Fenster überschreiten nie Fahrzeug-Grenzen). Für `predict()` wird das aktivste Fahrzeug mit jüngstem `timestamp.max()` als Default ausgewählt; dessen Daily-Slice wird in `self.daily` persistiert. Das 7×24-Stundenprofil mittelt mit gleichem Gewicht über alle Fahrzeuge, damit langlebigere Zeitreihen nicht dominieren. Erlaubt einen einzigen Forecaster pro Multi-Vehicle-Quelle (`emobpy`, `ved`).

2026-05-26 — Run-Verzeichnis-Schema Jede Forecast-Erzeugung schreibt in `predictions/<modell>/<datensatz>_<N>d_T<DD-MM-YYYY>/`. Ein Run ist damit ein vollständiges Verzeichnis-Objekt (CSV, Plots, Notizen) und kann über das Frontend in den Store importiert werden. Hinweis: Doppelpunkte sind in Windows-Pfaden verboten, deshalb `T<DD-MM-YYYY>` mit Bindestrich statt `TDD:MM:YYYY`.

2026-05-26 — Forecaster-Familie pro Datenquelle Trainingsskripte `train_<source>_forecaster` für alle vier Quellen ergänzt. `real_world_ev` ersetzt das frühere `car_full_forecaster` unter konsistentem Namensschema `<source>_forecaster_<algo>.joblib`. Das alte `joblib` bleibt für Backward-Compat erhalten; ein Shim in `model_scripts/csv_forecaster.py` re-exportiert die Klasse unter dem alten Modulpfad, damit `joblib.load` weiterhin funktioniert.

1. Einleitung

Die Elektromobilität ist ein immer wichtiger werdendes Thema. Sie trägt zur Reduzierung von Treibhausgasemissionen bei, verringert die Abhängigkeit zu fossilen Brennstoffen und somit auch wirtschaftliche Abhängigkeiten. Gleichzeitig bietet sie eine neue Möglichkeit bei der Energieversorgung.

Die Möglichkeit bei der Energieversorgung ist hier besonders interessant. Fahrzeuge, die mit Strom betrieben werden, können nicht nur als Verbraucher betrachtet werden, sondern auch als Energiespeicher. Elektrofahrzeuge können also nicht nur Energie aufnehmen, sondern diese auch wieder abgeben, was besonders im Kontext von erneuerbaren Energien von großer Bedeutung ist.

Im Moment bestehen noch kaum rentable Möglichkeiten, überschüssige Energie aus erneuerbaren Quellen zu speichern, weshalb dem Thema eine besondere Relevanz zukommt: Durch eine zuverlässige Speicherung könnte Energie auch dann genutzt werden, wenn Solar- oder Windanlagen nicht aktiv erzeugen. Damit ließe sich der Anteil erneuerbarer Energien an der Versorgung deutlich erhöhen, was ein wichtiger Beitrag zur Energiewende wäre.

Jedoch bringt eine Speicherung von Energie in Elektrofahrzeugen einen geringfügigen Mehrwert - ökonomisch wie auch ökologisch - wenn diese nicht zuverlässig gesteuert wird. Es wird ein System benötigt, das die intelligente Steuerung von Energieflüssen zwischen Fahrzeug und Stromnetz ermöglicht. Eine intelligente Steuerung ist nur möglich, wenn das Nutzerverhalten identifiziert werden kann. Es muss also ein System entworfen werden, das eine zuverlässige Vorhersage über das Mobilitätsverhalten des Nutzers ermöglicht.

1.1. Motivation

In dieser Arbeit wird untersucht, inwiefern sich das Nutzungsverhalten von Elektrofahrzeugen vorhersagen lässt, um potentiell den Energiefluss zwischen Fahrzeug und Stromnetz zu optimieren. Voraussetzung hierfür ist die Verfügbarkeit von V2G (Vehicle-to-Grid), also der Fähigkeit von Elektrofahrzeugen, Energie nicht nur aufzunehmen, sondern auch wieder ans Stromnetz abzugeben. Obwohl V2G zunehmend an Präsenz gewinnt, bestehen in der praktischen Anwendung weiterhin große Lücken. Ein zentraler Baustein für das effiziente Zusammenwirken von Elektrofahrzeugen

und Stromnetz ist ein zuverlässiges Vorhersagemodell des Mobilitätsverhaltens: Es erlaubt, Lade- und Entladezyklen so zu steuern, dass sowohl die Effizienz der mobilen Akkumulatoren als auch die Auslastung des Netzes optimiert werden. Entscheidend ist dabei die Zuverlässigkeit der Vorhersage, um ein fehlerloses System gewährleisten zu können.

1.2. Problemstellung

Die größte Hürde stellt eine stabile Datengrundlage dar. Da der Anwendungsfall sehr anspruchsvoll und dynamisch ist, lässt er sich mit bestehenden Ansätzen bislang nur unzureichend abbilden. Typischerweise erzielen Machine Learning Modelle sehr gute Ergebnisse bei dieser Art von Vorhersageproblemen. Voraussetzung dafür ist allerdings eine hochqualitative Datenbasis, denn Fehleinschätzungen können zu einer fehlerhaften Energieverwaltung und im schlimmsten Fall zur Nichtverfügbarkeit des Fahrzeugs führen. Es müssen also verlässliche Daten über das Mobilitätsverhalten erhoben oder von Dritten bezogen werden. Da Qualität in diesem Bereich nicht genau definiert ist, gilt es dies an bestimmten Kriterien und Werten festzulegen.

In Betrachtung der existierenden wissenschaftlichen Literatur besteht im Moment ein Mangel an Studien, die sich explizit mit dem Mobilitätsverhalten von Individuen und der Erstellung von Prognosemodellen für diesen Zweck beschäftigen. Es gibt wertvolle Studien, die sich mit dem Mobilitätsverhalten von Gruppen beschäftigen, jedoch liefern diese wenig Aufschluss darüber, wie sich der Individualverkehr vorhersagen lässt. Damit künftig eine intelligente Steuerung von Energieflüssen möglich wird, müssen zuverlässige Vorhersagemodelle entwickelt werden, die auf hochqualitativen Daten zum individuellen Mobilitätsverhalten basieren.

Bisherige Arbeiten zur Prognose im Mobilitäts- und Energiekontext konzentrieren sich überwiegend auf den Ladezeitpunkt, die Leistung oder den letztendlichen Ladezustand sowie auf aggregiertes Nutzungsverhalten ganzer Gruppen. Das individuelle Fahr- und Nutzungsverhalten einzelner Fahrzeuge wird dabei kaum modelliert, obwohl gerade dieses für eine genaue Steuerung bzw. Vorhersage von Nöten ist. Wo individuelles Fahrverhalten betrachtet wird, ist es zwischen den Fahrern stark heterogen, sodass ein einzelnes, übergreifendes Modell nicht den kompletten Kontext erfasst; zugleich ist die Datenbasis der wenigen einschlägigen Studien häufig nicht ausreichend oder – wie bei synthetisch erzeugten Profilen – nicht zuverlässig repräsentativ für reale Mobilitätsdaten. Eine ausführliche Einordnung der relevanten Forschungsstränge und die daraus abgeleitete Forschungslücke finden sich in Abschnitt 2.4.

1.3. Zielsetzung

Das Ziel der Arbeit ist es, zuverlässige Modelle zur Prognose für individuelles Mobilitätsverhalten zu entwickeln. Die Modelle sollen den Zweck erfüllen, genauer Uhrzeiten vorhersagen zu können, wann ein Fahrzeug fährt und wann es parkt. Beispielsweise soll ein Modell zwischen ein und sieben Tage lange Prognosen erstellen können. Dazu sind die erhobenen Daten qualitativ zu bewerten und wenn nötig, aus Datenschutzgründen zu anonymisieren. Auf dieser Grundlage werden Modelle erstellt, die das Mobilitätsverhalten von Nutzern möglichst genau vorhersagen können. Des Weiteren soll Gegenstand der Arbeit sein, die Vorhersagegenauigkeit verschiedener Modelle zu vergleichen und deren Einflussfaktoren zu quantifizieren. Die Ergebnisse sollen dazu beitragen, eine intelligente Steuerung von Energieflüssen zwischen Fahrzeug und Stromnetz zu ermöglichen, um so die Nutzung von erneuerbaren Energien zu verbessern und eine wichtige Rolle bei der Energiewende zu spielen.

1.4. Forschungsfragen

1. Wie zuverlässig lässt sich das individuelle Mobilitätsverhalten einzelner Fahrzeuge über einen Horizont von bis zu sieben Tagen vorhersagen?
 - a) Wie unterscheiden sich verschiedene Verfahren des maschinellen Lernens hinsichtlich ihrer Vorhersagegenauigkeit?
 - i. Welchen Einfluss hat die Datenqualität bzw. -quelle auf die Prognosegüte – getrennt vom Einfluss des gewählten Algorithmus?
 - b) Welche Merkmale beeinflussen das individuelle Mobilitätsverhalten am stärksten?
2. Welche Anforderungen an die Datengrundlage ergeben sich daraus, und wie lässt sich ihre Qualität bewerten?
 - a) Wie wirken sich Aufbereitung und eine aus Datenschutzgründen notwendige Anonymisierung auf die Nutzbarkeit der Daten aus?

1.5. Aufbau der Arbeit

TODO: nochmal überschauen/bearbeiten*

Die vorliegende Arbeit ist wie folgt aufgebaut:

Kapitel 1 führt in das Thema ein und fasst den Kerninhalt zusammen.

Kapitel 2 erläutert die theoretischen Grundlagen, die für das Verständnis dieser Arbeit notwendig sind.

Kapitel 3 beschreibt die gewählte Methodik und das Vorgehen.

Kapitel 4 stellt die Implementierung der entwickelten Lösung vor.

Kapitel 5 präsentiert die Evaluation und die erzielten Ergebnisse.

Kapitel 6 diskutiert die Ergebnisse und deren Bedeutung.

Kapitel 7 fasst die Arbeit zusammen und gibt einen Ausblick auf zukünftige Arbeiten.

2. Theoretische Grundlagen

Dieses Kapitel legt die Grundlagen, auf denen die folgende Methodik aufbaut; Sowohl Begrifflichkeiten, als auch thematische Grundlagen werden erläutert. Als erstes wird festgelegt, was genau Mobilitätsdaten sind und wann sie relevant sind (Abschnitt 2.1). Darauf folgt eine kurze Erläuterung zu Machine Learning und wieso die Verwendung in dieser Arbeit unerlässlich ist. Der vorletzte Punkt betrachtet, wie Mobilitätsdaten verarbeitet werden und anhand welcher Kriterien sich ihre Qualität bewerten lässt (Abschnitt 2.3). Als letztes ordnet Abschnitt 2.4 den aktuellen Stand der Forschung ein und arbeitet die Forschungslücke heraus, von der sich diese Arbeit in Abschnitt 2.4.1 abgrenzt.

2.1. Mobilitätsdaten

```
datetime,in_use,event,soc_pct,volt_v,curr_a,temp_c
2019-12-13 21:00:00,1,driving,58.8,412.8,-98.64,23.31
2019-12-13 22:00:00,1,driving,85.0,441.4,-47.771,27.52
2019-12-13 23:00:00,1,driving,86.0,436.76,37.899,28.5
2019-12-14 00:00:00,0,parked,86.0,436.76,37.899,28.5
2019-12-14 01:00:00,0,parked,86.0,436.76,37.899,28.5
2019-12-14 02:00:00,0,parked,86.0,436.76,37.899,28.5
2019-12-14 03:00:00,0,parked,86.0,436.76,37.899,28.5
2019-12-14 04:00:00,0,parked,86.0,436.76,37.899,28.5
```

(a) Datensatz: Cars Real World Electric (Pozzato u. a. 2023)

```
datetime,in_use,location,distance_km,consumption_kwh
2020-01-01 00:00:00,0,home,0.0,0.0
2020-01-01 01:00:00,0,home,0.0,0.0
2020-01-01 02:00:00,0,home,0.0,0.0
2020-01-01 03:00:00,0,home,0.0,0.0
2020-01-01 04:00:00,0,home,0.0,0.0
2020-01-01 05:00:00,0,home,0.0,0.0
2020-01-01 06:00:00,1,driving,15.0,3.377
```

(b) Datensatz: emobpy (Gaete-Morales u. a. 2020)

Abbildung 2.1.: Beispielhafte Auszüge zweier Mobilitätsdatensätze.

Wenn von Mobilitätsdaten gesprochen wird, ist im digitalen Rahmen meist die Rede von Dateien, die Informationen in chronologischer Abfolge enthalten. Diese Daten können in den unterschiedlichsten Formaten auftreten – mit beliebig vielen Einträgen und beliebig vielen Informationen pro Eintrag – und sind nicht zwingend homogen. Für

das individuelle Fahr- und Nutzungsverhalten einzelner Fahrzeuge existiert – anders als für ÖPNV- oder Sharing-Daten – kein etabliertes Standardformat. Daraus folgt, dass sich Datensätze aus verschiedenen Studien erheblich in Schema und Granularität unterscheiden: In Abbildung 2.1a finden sich neben einem üblichen Zeitstempel nur die physikalischen Messwerte und kaum Informationen über das Fahrzeug oder den Zweck des Aufenthalts. Abbildung 2.1b enthält dagegen die Aktivität, gefahrene Distanz und den Energiekonsum. Für diese Arbeit sind jedoch nur wenige Felder unabdingbar – im Wesentlichen Zeitstempel und Fahr- bzw. Parkzustand –, und diese sind in beiden Datensätzen enthalten. Trotz ihres erheblichen Unterschieds sind daher beide valide und relevant.

2.2. Machine Learning

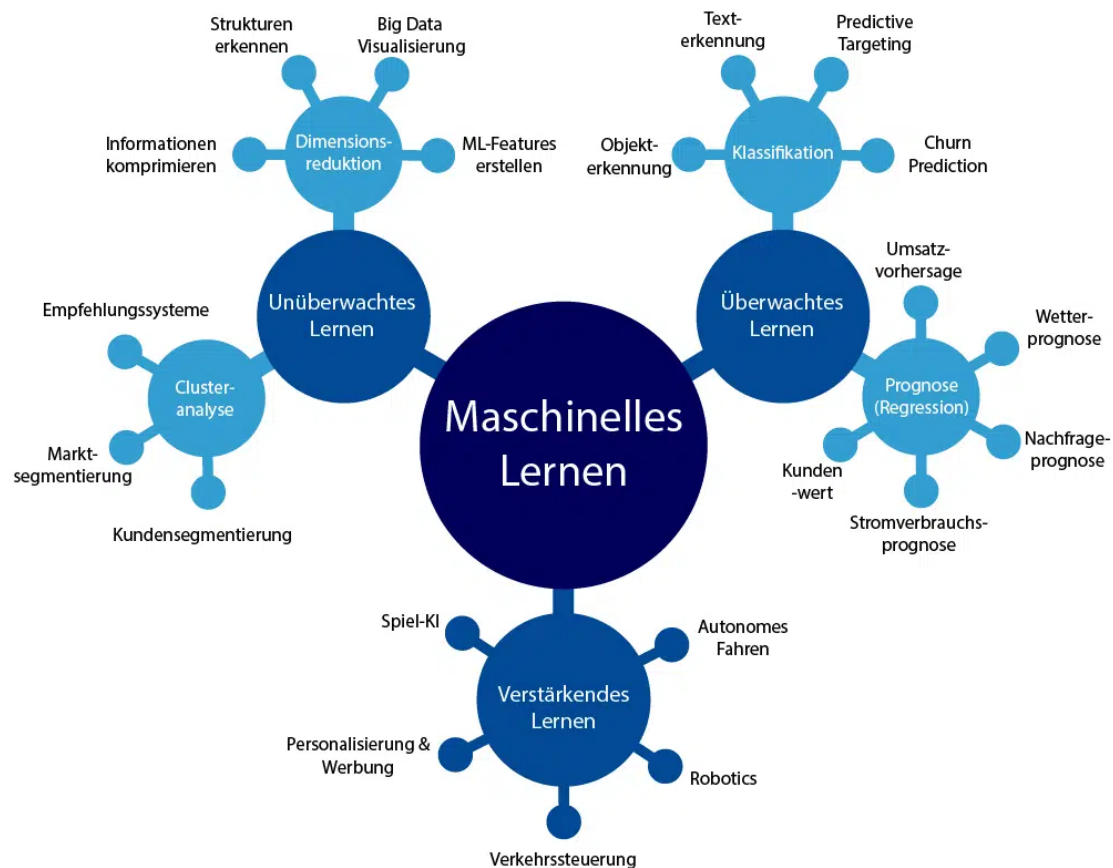


Abbildung 2.2.: Teilbereiche des maschinellen Lernens mit beispielhaften Anwendungen (Wuttke 2024).

Das Prinzip des maschinellen Lernens besteht darin, einem Computer ein gewünschtes Verhalten anhand von Beispieldaten beizubringen, anstatt es durch einen

fest vorgegebenen, regelbasierten Algorithmus explizit zu programmieren (Bishop 2006). Das Modell erkennt dabei statistische Muster in den Trainingsdaten und nutzt diese, um Vorhersagen für bislang ungesehene Eingaben zu treffen. Der Vorteil gegenüber einem starren Algorithmus liegt darin, dass komplexe Zusammenhänge – wie das individuelle Mobilitätsverhalten – erlernt werden können, ohne dass sie vorab vollständig bekannt sein müssen.

Häufig wird maschinelles Lernen mit künstlichen neuronalen Netzen gleichgesetzt, deren Aufbau vom menschlichen Gehirn inspiriert ist. Neuronale Netze bilden jedoch nur eine von vielen Verfahrensklassen. Die in dieser Arbeit eingesetzten Modelle – Random Forest und Gradient Boosting (Abschnitt 2.3 und Kapitel 3) – beruhen stattdessen auf Entscheidungsbäumen und sind nicht biologisch motiviert. **TODO: erweitern mit neuen modellen etc**

2.2.1. Überwachtes Lernen

In dieser Arbeit kommt ausschließlich überwachtes Lernen zum Einsatz (Hastie u. a. 2009). Dabei lernt das Modell aus Beispielpaaren von Eingaben und zugehörigen, bekannten Zielwerten. Je nach Art des Zielwerts unterscheidet man zwei Aufgabentypen, die in dieser Arbeit beide vorkommen: Klassifikation, bei der eine diskrete Klasse vorhergesagt wird (etwa ob ein Fahrzeug in einer gegebenen Stunde fährt oder geparkt ist), und Regression, bei der ein kontinuierlicher Wert geschätzt wird (etwa die Abfahrts- oder Rückkehrzeit). **TODO: korrigieren/überwachen: hauptmerkmal klassifikation, da bestimme minutenzahl erstmal uninteressant und somit nur stunden für fährt= 1 oder 0**

2.2.2. Merkmale und Feature Engineering

Bei Machine Learning ist oft die Rede von Feature Engineering, oder das umformen von Merkmalen (Géron 2022). Ein Merkmal ist beispielsweise eine Spalte in einer Comma-Separated-Value Datei (siehe Abbildung 2.1), hierbei könnte es sich um jede der Spalten handeln. Folglich ist der Zeitstempel, die Handlung, der Ort oder auch der Energiekonsum ein Merkmal bzw. Feature.

In der Regel sind die Daten nicht in ihrer Rohform nicht direkt aussagekräftig, weswegen sie interpretiert werden müssen. Ein Zeitstempel (Beispiel: 2026-05-25 08:00:00) hat ohne Kontext keinen bestimmten Wert, daher extrahiert man durch Feature Engineering die verborgenen Informationen und macht sie für das Modell zugänglich. Der genannte Zeitstempel bspw. ist im Kontext von Mobilität:

- Ein Montag -> Werktag
- Acht Uhr -> Pendlerzeit

Ein Modell entdeckt Muster nicht von selbst, daher muss man die Informationen verwertbar machen. Tage die nicht auf einen Werktag fallen, oder eine andere Uhrzeit haben, die keine Pendlerzeit ist, kann je nach restlichem Informationsgehalt (Ort, Fahrtstrecke, Aufenthaltsdauer) zu einem jeweils anderem Ergebnis führen.

Lag-Merkmal

Ein Lag ist der Wert eines Merkmals aus einem früheren Zeitschritt (Hyndman u. a. 2021). Lags sind teilweise periodisch und sind mit Stundenwerten gleichzusetzen. Ein **Lag_1** beispielsweise ist das selbe Event, nur eine Stunde in der Vergangenheit, selbiges Schema gilt für einen **Lag_168**, dieser liegt 168 Stunden in der Vergangenheit. Es gibt sehr spezifische Werte und diese erlauben auf bestimmte Muster Rückschlüsse. Lags von 1, 2, 24 und 168, wobei 24 Stunden einen Tag und 168 Stunden sieben Tage in der Vergangenheit abbilden.

Ein Risiko, das durch Lag auftreten kann, ist die Rechnung über mehr als ein Fahrzeug hinweg. Dadurch kann ein Verhalten auf ein anderes Fahrzeug interpretiert werden und falsche zugewiesene Verhaltensmuster errechnen.

Rolling-Aggregate

Ein „Rolling Mean“, oder Rolling-Mittel, fasst ein Zeitfenster zu einer Kennzahl zusammen (Hyndman u. a. 2021). So werden beispielsweise die letzten 24 Stunden oder 7 Tage Aktivitätsanteil zusammengefasst.

Data Leakage ist ein Risiko, das bei einem Rolling Mean auftreten kann, indem es die Zukunft einbezieht oder sogar den Zielwert selbst mit einrechnet. Dadurch lernt das Modell den tatsächlich gewünschten Wert und kann diesen, fälschlicherweise, richtig wiedergeben. Eine Falschimplementierung kann dazu führen, dass die Tests übermäßig gut abschneiden, im Realfall jedoch komplett versagen.

2.2.3. Training

Wenn ein Modell trainiert wird, passt es die eigenen internen Parameter an die Trainingsdaten an, sodass Vorhersagen möglichst gut zu den gelernten Zielwerten passt (Bishop 2006). Formal wird dabei das Resultat einer Verlustfunktion minimiert, welche misst, wie weit die Vorhersage von der Wahrheit abweicht. Bei einem baumbasierten Modell werden konkret Splits gewählt für bestimmte Merkmale und Schwellen, die gewisse Zielgrößen im jeweiligen Knoten am besten trennen.

Gini Index

Die allgemeine Form der Gini-Unreinheit für einen Knoten mit C Klassen und den Klassenanteilen p_i lautet (Breiman u. a. 1984):

$$G = 1 - \sum_{i=1}^C p_i^2 \quad (2.1)$$

Für den binären Fall mit zwei Klassen ($p_1 = p$, $p_0 = 1 - p$) vereinfacht sich Gleichung (2.1) zu (Breiman u. a. 1984):

$$G = 1 - (p^2 + (1 - p)^2) = 2p(1 - p) \quad (2.2)$$

Hier spricht man von einer Unreinheit: einer Metrik, die ausdrückt, mit welcher Wahrscheinlichkeit ein Wert einer falschen Klasse zugeordnet würde, wenn er anhand der Klassenverteilung im Knoten ein zufälliges Label erhielte. Die gewichtete Unreinheit eines Splits ergibt sich aus den Unreinheiten der beiden Kindknoten (Breiman u. a. 1984):

$$G_{\text{split}} = \frac{n_{\text{links}}}{n} G_{\text{links}} + \frac{n_{\text{rechts}}}{n} G_{\text{rechts}} \quad (2.3)$$

In Gleichung (2.3) zeigt n die Elemente im Elternknoten und G die Unreinheit des jeweiligen Knoten an. Der Rest funktioniert analog, anhand der vorausgegangenen Formeln. Mit der Split-Formel lassen sich mehrere Iterationen pro Knoten vornehmen, wovon dann der reinste Split verwendet wird, um im Baum eine Rekursionsebene tiefer zu schreiten. Insgesamt funktioniert dieser Split $n - 1$ mal.

2.2.4. Generalisierung

Das Prinzip der Generalisierung ist es, ein Modell so zu trainieren, dass es die Muster der Daten erkennt und deren Berechnung erlernt (Hastie u. a. 2009). Das bedeutet, dass das Modell auch mit neuen Daten – die ungesehene Inhalte verwenden – umgehen kann, die nah am gelernten Schema liegen. Tabelle 2.1 fasst zusammen, wie sich Unter-, gute und Überanpassung anhand von Trainings- und Testfehler unterscheiden lassen.

Overfitting

Bei einem überangepassten Modell sind Trainingsfehler niedrig und Testfehler hoch. Kurzum bedeutet das, dass das Modell die Daten zu gut kennt und diese quasi auswendig weiß, bei neuen Daten jedoch oft und stark abweicht.

Underfitting

Ein Modell das unterangepasst ist, schneidet weder beim Training, noch beim Test gut ab. Hier ist das Modell zu simpel bzw. seine Kapazität zu gering, um das zugrunde liegende Muster zu erfassen. Fehler bleiben dadurch bei beiden Datensätzen hoch.

Anpassung	Trainingsfehler	Testfehler	Charakteristik
Underfitting	hoch	hoch	Modell zu simpel; das zugrunde liegende Muster wird nicht erfasst.
Gute Anpassung	niedrig	niedrig	Kleine Generalisierungslücke; die Muster werden auf neue Daten übertragen.
Overfitting	sehr niedrig	hoch	Große Generalisierungslücke; Trainingsdaten werden inklusive Rauschen auswendig gelernt.

Tabelle 2.1.: Charakterisierung von Anpassungen anhand von Fehlerwerten Géron 2022.

2.2.5. Validierung

Durch die Validierung ist es möglich der Genauigkeit des Modells einen numerischen Wert zuzuweisen, was funktioniert, indem man das Modell Daten bewerten lässt, die bisher nicht bekannt waren.

$$CV(\hat{f}) = \frac{1}{N} \sum_{i=1}^N L(y_i, \hat{f}^{-\kappa(i)}(x_i)) \quad (2.4)$$

Die Gleichung (2.4) beschreibt die Genauigkeit eines Modells auf einer bestimmten Menge Daten. Hierfür wird ein Datensatz in K gleichgroße Mengen unterteilt – auch *Folds* genannt – gefolgt von $K + 1$ trainings. Da K *Folds* existieren, wird das Modell einmal mit jedem *Fold* getestet und hat zur Grundlage die restlichen *Folds*.

2.2.6. Bewertungsmetriken

2.2.7. Entscheidungsbäume

Sie bilden einen Teil des maschinellen Lernens, sind jedoch grundlegend verschieden zu den menschlich inspirierten neuronalen Netzen. Des Weiteren lassen sich Entscheidungsbäume nicht konkret in unsupervised oder supervised Modelle klassifizieren, denn die Struktur diktiert letztenendes, welchem Paradigma das Modell unterliegt.

2.2.8. Ensemble-Methoden: Random Forest und Gradient Boosting

2.2.9. Unsicherheit und Quantilsregression

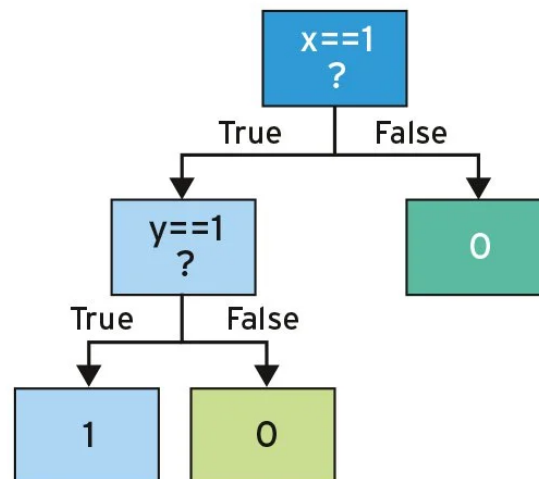


Abbildung 2.3.: Simpler binärer Entscheidungsbaum (Schilli 2017)

2.3. Verarbeitung von Mobilitätsdaten

Die Mobilitätsdaten bilden einen Hauptaspekt dieser Arbeit, da sie die Grundlage für das Training und die Validierung der Modelle darstellen. Abbildung 2.5 zeigt den Ablauf, den die Daten von der Recherche bis zur Nutzung durchlaufen; jeder der sechs Abschnitte umfasst dabei mehrere Teilschritte. Tabelle 2.2 fasst diese je Phase zusammen, die folgenden Absätze erläutern sie.

Recherche: Der erste Schritt ist die Identifikation geeigneter Datenquellen. Dabei ist zu beachten, dass Mobilitätsdaten allein noch keine Relevanz bedeuten: Die Daten müssen einer Studie entstammen, die für die individuelle Fahrzeugnutzungsprognose einschlägig ist. Erst nach hinreichender Prüfung dieser thematischen Eignung werden die Daten weiterverfolgt.

Beschaffung: Geeignete Daten müssen anschließend bezogen werden, durch Download offener Datensätze, Anfrage bei datenhaltenden Dritten oder eigene Erhebung. Bereits hier sind rechtliche und datenschutzrechtliche Rahmenbedingungen zu berücksichtigen, da personenbezogene Bewegungsdaten besonders schützenswert sind. Da jedoch der zeitliche Rahmen der Arbeit nicht für eigene Erhebung ausreicht, sind die Daten auf öffentlich zugängliche oder Kooperationen beschränkt.

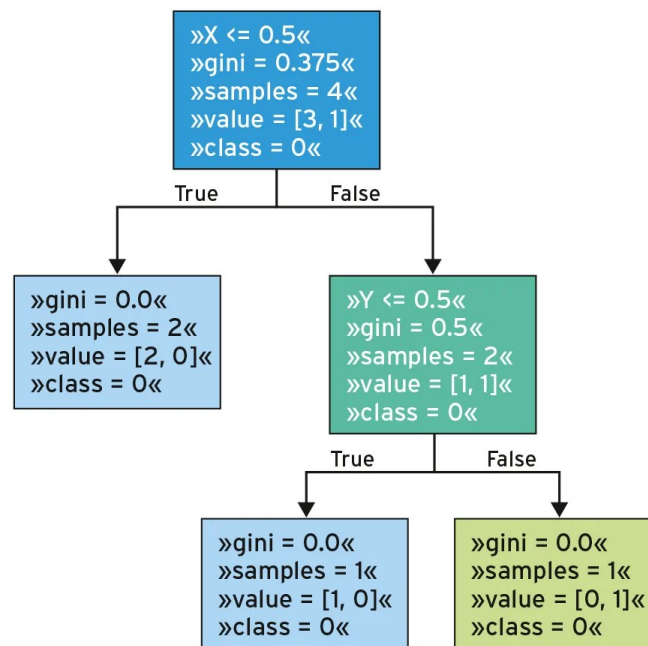


Abbildung 2.4.: Entscheidungsbaum mit Aufteilungskriterien (Gini-Index, Stichprobengröße und Klassenverteilung pro Knoten) (Schilli 2017)

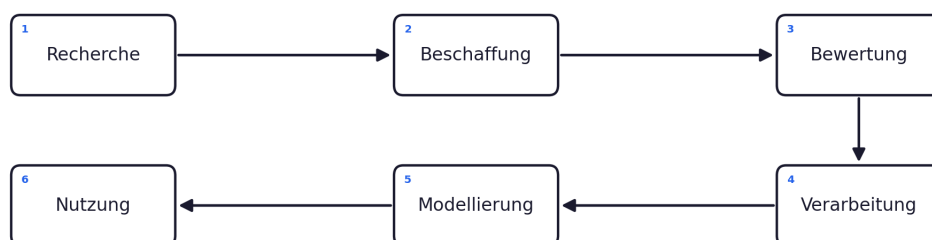


Abbildung 2.5.: Generalisierte Pipeline zur Verarbeitung von Mobilitätsdaten.

Phase	Wesentliche Teilschritte
1 – Recherche	Identifikation relevanter Studien und Datensätze; Prüfung der thematischen Eignung.
2 – Beschaffung	Bezug der Daten unter Beachtung rechtlicher und datenschutzrechtlicher Vorgaben.
3 – Bewertung	Beurteilung der Datenqualität anhand definierter Kriterien.
4 – Verarbeitung	Bereinigung, Überführung in ein einheitliches Schema, Feature-Engineering und gegebenenfalls Anonymisierung.
5 – Modellierung	Training und Validierung der Klassifikations- und Prognosemodelle.
6 – Nutzung	Anwendung der Modelle zur Vorhersage und Bereitstellung der Ergebnisse für nachfolgende Systeme.

Tabelle 2.2.: Phasen der Mobilitätsdaten-Pipeline

Bewertung: Vor der Weiterverarbeitung wird die Qualität der Daten beurteilt. Da der Begriff der Datenqualität in diesem Bereich nicht eindeutig definiert ist, wird er an konkreten Kriterien festgemacht: Vollständigkeit, zeitliche Auflösung und Umfangänglichkeit der Daten. Diese Bewertung entscheidet letztendlich darüber, ob ein Datensatz für ein zuverlässiges Modell geeignet ist.

Verarbeitung: Die als geeignet bewerteten Daten werden bereinigt und in ein einheitliches Schema überführt, sodass Quellen unterschiedlicher Herkunft vergleichbar werden. Anschließend werden aus den Rohdaten die für die Modelle benötigten Merkmale abgeleitet und bei Bedarf erfolgt zudem eine Anonymisierung.

Modellierung: Auf der aufbereiteten Datenbasis werden die Modelle trainiert und validiert. Dieser Schritt umfasst sowohl die Klassifikation des Fahrverhaltens als auch die eigentliche Prognose der Fahrzeugnutzung.

Nutzung: Abschließend werden die trainierten Modelle angewendet, um das individuelle Mobilitätsverhalten vorherzusagen. Die daraus gewonnenen Prognosen werden dokumentiert und bereitgestellt. Sie liefern die Grundlage für Vorhersagen in der Fahrzeugnutzung und damit ein weiterer Schritt hin zu einem intelligenten Energiesystem.

2.4. Stand der Forschung

Die Forschung zur Prognose individueller Fahrzeugnutzung lässt sich grob in mehrere Sektionen gliedern, die sich in Zielgröße, Methodik und Datengrundlage unterscheiden.

Tabelle 2.3 fasst diese Sektionen als Übersicht zusammen. Jeder dieser Sektionen beinhaltet Arbeiten, die zum Teil entweder relevant für diese Arbeit, oder zu Ihrer Eingrenzung dient. Die folgenden Unterabschnitte vertiefen die Sektionen jeweils und arbeiten am Ende die für diese Arbeit zentrale Forschungslücke heraus.

Forschungsstrang	Typische Methoden	Datengrundlage	Offene Punkte
Mobilitäts- und Nutzungsforecast	Statistische Modelle (ARIMA), Random Forest, neuronale Netze (LSTM)	Reale Telematik- und Bewegungsdaten	Generalisierung über Fahrzeuge und Quellen hinweg
Driving-/Parking-Klassifikation	Random Forest, SVM, Gradient Boosting	EV-Flottendaten, stündliche Profile	Einfluss der Datenqualität auf die Klassifikationsgüte
EV-Last- und Ladeprognose	Gradient Boosting, Deep Learning	Synthetische Daten (z. B. emobpy), reale EV-Daten	Vergleichbarkeit synthetischer und realer Datengrundlagen
Unsicherheits- und Quantilsprognose	Quantilsregression, Quantile-Random-Forest	Heterogene Mobilitätsdaten	Kalibrierung individueller Vorhersageintervalle (P10/P90)

Tabelle 2.3.: Zentrale Forschungssektionen im Umfeld der individuellen Fahrzeugnutzungsprognose.

TODO: Aktuellen Stand der Forschung zusammenfassen. Welche verwandten Arbeiten gibt es? Was wurde bereits untersucht? Pro Tabellenzeile eine Subsection mit synthetisierter Quellenlage anlegen, abschließend Forschungslücke formulieren.

2.4.1. Verwandte Arbeiten

TODO: Verwandte Arbeiten vorstellen und voneinander abgrenzen. Wie unterscheidet sich die eigene Arbeit?

- Forschungsarbeit/SUMO
- ähnliche Studien
-

2.4.2. Mobilitäts- und Nutzungsforecast

Ein zentraler Befund dieser Sektion betrifft die Heterogenität individuellen Fahrverhaltens. Ji u. a. (2023) werten über vier Millionen Fahrten von 3743 privaten Fahrzeugführern in sechs US-Metropolregionen aus und zeigen, dass sich die Fahrer in fünf deutlich verschiedene Typen gliedern lassen (Tabelle 2.4), die sich vor allem in ihrer Pendelregelmäßigkeit und im Anteil nicht-arbeitsbezogener Fahrten unterscheiden. Diese Heterogenität erklärt, warum ein einzelnes, übergreifendes Modell dem Individualverkehr nur begrenzt gerecht wird, und motiviert eine individuelle bzw. quellspezifische Modellierung.

Fahrertyp	Charakterisierung
Homebodies	Wenigfahrer, die nur kurz und selten das Haus verlassen; geringe Mobilität.
Gig Drivers	Fahrer mit hohem, auftragsgetriebenem Fahraufkommen (z. B. Liefer- und Fahrdienste) ohne festen Arbeitsort.
Movers	Individuen mit breit gestreutem Fahrverhalten und vielen wechselnden Zielen, ohne klares Muster.
Typical Drivers	Durchschnittsfahrer mit einer Mischung aus Arbeits- und Freizeitfahrten und mittlerer Regelmäßigkeit.
Work-focused Commuters	Berufspendler mit stark regelmäßigem Pendelmuster, wenig nicht-arbeitsbezogenen Fahrten und hoher Vorhersagbarkeit.

Tabelle 2.4.: Fünf Fahrertypen des Individualverkehrs nach Ji u. a. (2023).

2.5. Zusammenfassung

TODO: Kurze Zusammenfassung der wichtigsten Erkenntnisse aus diesem Kapitel.

3. Methodik

TODO: Einleitenden Text verfassen. Warum wurde die gewählte Methodik verwendet?

- verschiedene modelle gegenüberstellen
- finetuning von modellen
- datenlage
- preprocessing von daten
- behandlung von sonderbaren werten (entfernen, stärker repräsentieren, etc.)
- robustheit von modell testen/erhöhen
- ausgesuchtes netz anpassen, architektur anpassen, etc.
- erklärung für modelle, versuche usw

3.1. Forschungsdesign

TODO: Das übergeordnete Forschungsdesign beschreiben (z. B. qualitativ, quantitativ, Mixed-Methods, Design Science Research, etc.).

3.2. Vorgehen

TODO: Detailliertes Vorgehen beschreiben. Welche Schritte wurden in welcher Reihenfolge durchgeführt?

3.3. Datenerhebung

TODO: Wie wurden die Daten erhoben? Welche Quellen, Werkzeuge oder Methoden wurden eingesetzt?

3.4. Datenanalyse

TODO: Wie wurden die erhobenen Daten analysiert? Welche Analysemethoden kommen zum Einsatz?

3.5. Zusammenfassung

TODO: Kurze Zusammenfassung der Methodik.

4. Implementierung

TODO: Einleitenden Text verfassen. Was wird in diesem Kapitel vorgestellt?

4.1. Systemarchitektur

TODO: Die Gesamtarchitektur des Systems beschreiben. Welche Komponenten gibt es und wie interagieren sie?

4.2. Technologieauswahl

TODO: Welche Technologien, Frameworks, Programmiersprachen und Werkzeuge wurden eingesetzt und warum?

4.3. Umsetzung

TODO: Zentrale Aspekte der Implementierung beschreiben. Wichtige Algorithmen, Datenstrukturen oder Designentscheidungen erläutern.

4.4. Herausforderungen

TODO: Welche Herausforderungen traten bei der Implementierung auf? Wie wurden sie gelöst?

4.5. Zusammenfassung

TODO: Kurze Zusammenfassung der Implementierung.

5. Evaluation

TODO: Einleitenden Text verfassen. Was wird evaluiert und mit welchem Ziel?

5.1. Evaluationsdesign

TODO: Das Design der Evaluation beschreiben. Welche Metriken, Kriterien oder Hypothesen werden überprüft?

5.2. Testumgebung

TODO: Die Testumgebung beschreiben (Hardware, Software, Datensätze, Konfigurationen).

5.3. Ergebnisse

TODO: Die Ergebnisse der Evaluation präsentieren. Tabellen und Grafiken verwenden.

5.4. Zusammenfassung

TODO: Kurze Zusammenfassung der Evaluationsergebnisse.

6. Diskussion

TODO: Einleitenden Text verfassen. Was wird in diesem Kapitel diskutiert?

6.1. Interpretation der Ergebnisse

TODO: Die Ergebnisse aus Kapitel 5 interpretieren. Was bedeuten die Ergebnisse im Kontext der Forschungsfragen?

6.2. Vergleich mit verwandten Arbeiten

Da sich bisher keine anderen Arbeiten mit der Erfassung und Erstellung von Modellen für die Prognose von individuellem künftigem Mobilitätsverhalten befassen, lag keine Grundlage für einen direkten Vergleich vor und kann somit keine Einschlägige Aussage tätigen.

Studien die sich mit individuellem Mobilitätsverhalten befassen, schauen hauptsächlich nach *State of Charge* oder *Ortsdaten*. **TODO:** Wie verhalten sich die Ergebnisse im Vergleich zu verwandten Arbeiten aus Abschnitt 2.4.1? – – verbessern

6.3. Limitationen

TODO: Welche Einschränkungen hat die Arbeit? Was konnte nicht untersucht werden? Welche Annahmen wurden getroffen? – – verbessern

- aktive datenerhebung
- realer verkehr
- parallele studien

6.4. Implikationen

TODO: Welche praktischen und theoretischen Implikationen ergeben sich aus den Ergebnissen?

7. Fazit und Ausblick

7.1. Zusammenfassung

TODO: Die gesamte Arbeit kurz zusammenfassen. Welche Forschungsfragen wurden beantwortet? Was sind die zentralen Ergebnisse?

7.2. Beantwortung der Forschungsfragen

TODO: Jede Forschungsfrage aus Abschnitt 1.4 explizit beantworten.

1. **TODO:** Antwort auf Forschungsfrage 1
2. **TODO:** Antwort auf Forschungsfrage 2
3. **TODO:** Antwort auf Forschungsfrage 3

7.3. Ausblick

TODO: Welche zukünftigen Arbeiten könnten auf dieser Arbeit aufbauen? Welche offenen Fragen bleiben bestehen?

Literaturverzeichnis

- Bishop, Christopher M. (2006). *Pattern Recognition and Machine Learning*. New York: Springer.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen und Charles J. Stone (1984). *Classification and Regression Trees*. Belmont, CA: Wadsworth.
- Gaete-Morales, Carlos, Hendrik Kramer, Wolf-Peter Schill und Alexander Zerrahn (2020). *emobpy: A Tool for Generating Battery Electric Vehicle Time Series*. Synthetische BEV-Profile auf Basis deutscher Mobilitätsstatistik (MiD). DOI: 10.5281/zenodo.3931663.
- Géron, Aurélien (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3. Aufl. Sebastopol, CA: O'Reilly Media.
- Hastie, Trevor, Robert Tibshirani und Jerome Friedman (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2. Aufl. New York: Springer. DOI: 10.1007/978-0-387-84858-7.
- Hyndman, Rob J. und George Athanasopoulos (2021). *Forecasting: Principles and Practice*. 3. Aufl. Melbourne: OTexts. URL: <https://otexts.com/fpp3/> (besucht am 15.06.2026).
- Ji u. a. (2023). „Rethinking the Regularity in Mobility Patterns of Personal Vehicle Drivers: A Multi-city Comparison Using a Feature Engineering Approach“. In: *Transactions in GIS* 27.3, S. 663–685. DOI: 10.1111/tgis.13043.
- Pozzato, Gabriele, Anirudh Allam und Simona Onori (2023). *Data from: Analysis and Key Findings from Real-World Electric Vehicle Field Data*. Reale 1-Jahres-Felddaten eines Audi e-tron (CAN-Bus, SOC). DOI: 10.17632/7vdkzpnjgj.2. URL: <https://data.mendeley.com/datasets/7vdkzpnjgj/2> (besucht am 13.06.2026).
- Schilli, Michael (2017). „KI lernt mit Entscheidungsbaeumen“. In: *Linux-Magazin* 08. URL: <https://www.linux-magazin.de/ausgaben/2017/08/snapshot/> (besucht am 13.06.2026).
- Wuttke, Laurenz (2024). *Machine Learning: Algorithmen, Methoden und Beispiele*. Datasolut GmbH. URL: <https://datasolut.com/was-ist-machine-learning/> (besucht am 13.06.2026).

A. Anhang

A.1. Zusätzliche Abbildungen

TODO: Zusätzliche Abbildungen hier einfügen, falls vorhanden.

A.2. Zusätzliche Tabellen

TODO: Zusätzliche Tabellen hier einfügen, falls vorhanden.

A.3. Quellcode

TODO: Relevanten Quellcode hier einfügen, falls vorhanden.

Eidesstattliche Erklärung

Ich versichere hiermit, dass ich die vorliegende Masterarbeit mit dem Titel

Titel der Masterarbeit

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder unveröffentlichten Schriften entnommen sind, sind als solche kenntlich gemacht. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen.

.....
Ort, Datum

.....
Unterschrift